

Prompt Engineering

Best Practices for Working with AI Language Models

Prompt Design

Structured Prompts

Parameters

Code Generation

What We'll Cover

01

Prompt Design

Elements, structure & crafting effective prompts

02

Structured Prompts

Zero-shot, Few-shot, Chain of Thought & more

03

LLM Parameters

Temperature, Top-P, Top-K, Max Tokens & Seed

04

Generating Code

Best practices for AI-assisted code generation

What is a Prompt?

The primary interface between humans and AI models

A prompt is **a means to interact with AI models** to accomplish a task — providing context, instructions, and guidance to generate meaningful responses.

Input Types

- Instructions & tasks
- Questions & queries
- Context & examples
- Constraints & requirements

Purpose

- Guide model behaviour
- Specify output format
- Set tone & style
- Provide domain context

Quality Impact

- Response relevance
- Accuracy of output
- Format correctness
- Reduction of hallucinations

Elements of a Prompt

Every effective prompt is built from four core components

01 Instruction

A specific task or action you want the model to perform.

Example: "Summarize the following paragraph in 3 bullet points."

02 Context

External information that steers the model toward better, more targeted responses.

Example: "You are a senior software engineer reviewing code."

03 Input Data

The actual input or question the model needs to address.

Example: "Text: The quick brown fox jumps over the lazy dog."

04 Output Indicator

Specifies the desired type or format of the response.

Example: "Respond in JSON format with keys: summary, keywords."

Prompt Design Best Practices



Clear Objective

Define what you want before writing. Align the prompt with your desired outcome.



Specific Instructions

Include details. Use persona, delimiters, and desired output length.



Sufficient Context

Explain the scenario, provide reference text, and give examples when possible.



Split Complex Tasks

Break multi-step goals into subtasks. Use step-by-step instructions.



Model Time to Think

Ask the model to reason before answering. Use 'think step by step'.



Define Output Format

Specify JSON, bullet points, table, or paragraph. Include edge cases (IDK).

Structured Prompts

Zero-shot · Few-shot · Chain of Thought · Self-Consistency · ReAct & more

Zero-Shot vs Few-Shot Prompting

Zero-Shot

No examples — model uses its training knowledge alone.

```
Classify the sentiment of:  
"I loved the product!"  
  
Sentiment:
```

- ✓ Works out-of-the-box
- ✓ Fastest to write
- ✗ Less control over output

Few-Shot

Provide 2–5 examples to guide the model's pattern.

```
Text: "Terrible!"  
Sentiment: Negative  
  
Text: "Loved it!"  
Sentiment:
```

- ✓ Higher accuracy
- ✓ Controls output format
- ✗ Costs more tokens

Chain of Thought (CoT) Prompting

Enable step-by-step reasoning for complex problems

What is CoT?

- Uses sequential prompts to build reasoning across steps.
- Asks the model to show its work before answering.
- Dramatically improves accuracy on math, logic, and multi-step tasks.
- Trigger: add "Think step by step" to your prompt.
- Works best with large, capable models.

Example

Q: Roger had 5 balls. He bought
2 more cans of 3 balls each.
How many balls does he have?

A (Standard): 11

A (CoT): Roger starts with 5.
2 cans $\times$ 3 balls = 6 balls.
5 + 6 = 11 balls.

Answer: 11

More Prompting Techniques

Self-Consistency

Sample multiple reasoning paths and take the majority vote answer for higher reliability.

ReAct Prompting

Interleave reasoning traces with task-specific actions (search, calculate) using LLMs.

Directional Stimulus

Provide hints or nudges to steer the model toward a desired direction or answer style.

Automatic Prompt Eng.

Use another AI to generate or optimize prompts — let LLMs write prompts for LLMs.

Generative Knowledge

Ask the model to generate relevant facts or knowledge before answering the main question.

Graph Prompting

Encode complex relationships using graph structures to improve structured reasoning tasks.

LLM

Parameters

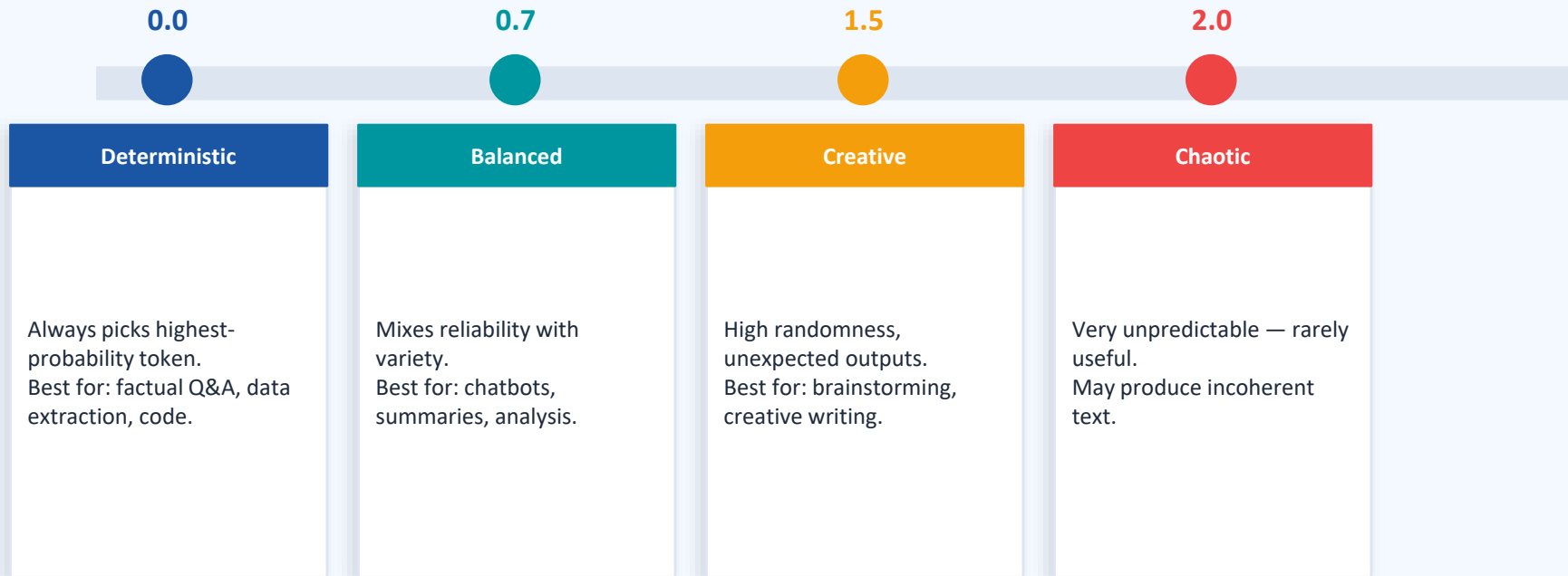
Temperature · Top-P · Top-K · Max Tokens · Seed

Key LLM Parameters at a Glance

Parameter	Range	Default	Effect
Temperature	0.0 – 2.0	1.0	Controls randomness / creativity
Top-P	0.0 – 1.0	1.0	Nucleus sampling — token probability mass
Top-K	1 – vocab	–	Limits candidate tokens per step
Max Tokens	1 – limit	4096	Hard cutoff for output length
Seed	Integer	None	Reproducibility — same seed = same output
Stop Sequences	Strings	None	Tokens that terminate generation early

Temperature

The most influential creativity knob



💡 For most factual or code tasks, use temperature = 0.

Top-P & Top-K

Two complementary sampling strategies

Top-P (Nucleus Sampling)

- Considers only tokens whose cumulative probability $\geq$ `top_p`.
- `top_p = 0.1` $\rightarrow$ only top 10% probability tokens considered.
- `top_p = 1.0` $\rightarrow$ all tokens considered (no filtering).
- Alternative to temperature — do not use both.

```
top_p = 0.1 # factual
top_p = 0.9 # diverse
```

Top-K

- Limits the pool to the K most-likely next tokens.
- `top_k = 1` $\rightarrow$ greedy decoding (always most likely).
- `top_k = 40` is a common default for chat models.
- Smaller K $\rightarrow$ more predictable; larger K $\rightarrow$ more variety.

```
top_k = 1    # deterministic
top_k = 40   # balanced
```

Max Output Tokens & Seed

Max Output Tokens

- Sets a hard upper limit on generated tokens.
- Does NOT guarantee a certain length — model may stop earlier.
- Model stops at: natural end, stop sequence, or limit hit.
- Reduce for cost control and faster responses.
- Increase for long reports, code files, or essays.
- 1 token $\approx$ 0.75 words in English.

Seed (Reproducibility)

- Integer that initialises the random number generator.
- Same seed + same prompt $\rightarrow$ same (or very similar) output.
- Critical for: testing, debugging, demos, auditing.
- Different seeds produce different creative outputs.
- Supported by OpenAI, Google Gemini, and others.
- Not a substitute for determinism — temp = 0 is better.

```
max_tokens=500, seed=42 → reproducible, cost-efficient output
```

Generating Code

Best practices for AI-assisted code generation

Prompting for Code — Best Practices

Specify Language & Framework

Always state the language, version, and relevant libraries upfront. "Python 3.11 using FastAPI"

Describe Input & Output

Explain what data goes in, what should come out, and the data types. Prevents ambiguous solutions.

Include Edge Cases

Mention error handling, null values, boundary conditions. Ask for try/except and input validation.

Request Comments & Docs

Ask for docstrings, type hints, and inline comments. "Add detailed docstrings and type annotations."

Ask for Tests

Request unit tests alongside the code. "Write pytest unit tests for each function."

Iterate & Refine

Review output, ask follow-up questions. "Refactor for readability" or "optimize for performance".

Prompt Example — Writing Code

✗ Less Effective

```
Write a function to  
parse a CSV file.
```

- ✗ No language specified
- ✗ No input/output described
- ✗ No error handling requested
- ✗ No format or style guidance

✓ Better

```
Write a Python 3.11 function  
that reads a CSV and returns  
a list of dicts. Handle missing  
fields and add docstrings.
```

- ✓ Language & version specified
- ✓ Input/output clearly defined
- ✓ Edge cases mentioned
- ✓ Documentation requested

Key Takeaways



Prompt Design

Use all four elements: Instruction, Context, Input Data, Output Indicator.



Structured Prompts

Match the technique to the task — CoT for reasoning, Few-shot for pattern control.



Parameters

Start with temp=0.7; tune Top-P or Top-K for diversity; fix Seed for reproducibility.



Code Generation

Specify language, I/O, edge cases, and ask for tests and documentation.

Thank You